Q: Object Inspection and its types

A: An object inspector is a GUI window associated with a simulation object

– Displays contents and properties

Three types

– Network Display

– Log Viewer

– Object Inspector

Q: Computer Models
A: The model definition could be

- Descriptive
- Analytical
- Mathematical
- Algorithmic

The computer models can be described into various types

- Stochastic vs Deterministic
- Continuous vs discrete
- Steady state vs Dynamic
- Local or Distributed
- Linear or nonlinear
- Open or closed

Q: Difference between Stochastic vs Deterministic?

A: A deterministic model always gives the same output for the same input, with no randomness—it's used in systems where behavior is predictable, like physics or engineering. A stochastic model includes randomness, so results can vary; it's used when uncertainty is important, like in weather forecasting, stock market analysis, or population studies. Deterministic models are simpler and faster, while stochastic models are better for simulating real-world variability.

Q: Components of a simulator and types of simulations.

A: Components of a simulator

- A self-contained program
- Event queue
- Simulation clock
- State variables

- Event routines
- Input routine
- Report generation routine
- Initialization routine
- Main program

Types of simulations

- Monte Carlo simulation
- Trace driven
- Discrete events
- Continuous events

Q: Continuous event and Discrete event?

A: In network modeling, discrete events happen at specific times (like packet arrivals), and they usually involve a finite number of events and outputs, making them easier to simulate and analyze. In contrast, continuous events change smoothly over time (like signal strength), and can involve infinite possible states or outputs since values change constantly. Discrete models are event-driven, while continuous models are time-driven.

Q: What is OMNET++?
A:

- Objective Modular Network Testbed in C++
    - Simulation kernel
    - Component-based simulation library
- A framework, not a simulator
- Designed to create & simulate any network

OMNeT++ stands for Objective Modular Network Testbed in C++. It is a framework, not just a simulator, used to create and simulate all types of networks. It has a simulation kernel and a component-based simulation library, which means you can build networks using reusable modules. OMNeT++ is mainly used for simulating communication networks like the internet or wireless systems. It uses C++ for coding behavior and NED (Network Description) language for designing network structures. It helps test how data moves, how protocols work, and how networks perform over time.

Q: Write down supported DNS operations.
A:

DNS Support in OMNET++

- Extensions provide classes and functions to simulate DNS and MDNS traffic

- Implement RFC 1035

Supported DNS Operations

- Name servers with recursive resolving capabilities
- Authoritative servers with DNS zone configuration using master files
- Caching servers without zones
- Only recursively resolving
- DNS Cache base that can be extended
- Caches based on different policies possible
- DNS client that can query a DNS server

Q: Random Request Mode and Scripted Mode.
A: Random request mode is when the browser generates requests to random web servers using statistical distributions. This means the requests happen in a random pattern based on probabilities.

Scripted mode is when the browsing follows a fixed list of predefined websites to visit at specific times. The behavior is planned and repeats exactly as scripted.

Q: Health Metrics

A:

Health Metrics in Network Monitoring

Incoming Traffic Variation (ITV)

ITV measures how the incoming bandwidth consumption changes across different sessions. It is calculated as the difference between the current session's bandwidth (IBC_1) and the previous session's bandwidth, divided by the session length. The function q(x) evaluates the variation, where higher values of x reduce the score exponentially. If the variation is negative or zero, the score remains 1, indicating stable traffic.

Traffic Tolerance (TT)

Traffic Tolerance (TT) evaluates the Round Trip Time (RTT) between the end-user and the ISP's recursive resolver. The function q(x) decreases linearly when RTT is within acceptable limits (up to twice the average RTT), and drops to 0 if RTT exceeds that threshold. This metric helps assess how well the network handles delay-sensitive applications.

DNS Requests per Second (DNSR)

DNSR indicates how many DNS queries are made per second in a session. The function q(x) decreases linearly as the number of queries increases, up to twice the average DNSR. If the queries exceed that range, the score becomes 0. This metric helps detect excessive DNS traffic, which could suggest misbehavior or attacks.

Rate of Repeated Queries (RRQ)

RRQ measures how often the same DNS name is requested more than once in a session, which usually happens if the initial response is lost or not cached. Ideally, names should only be resolved once per session due to caching. The function $q(x) = 1 - x / R_{max}$ assigns a lower score to sessions with more repeated queries, indicating poor caching or packet loss.

Incoming Bandwidth Consumption (IBC)

IBC measures the ratio between the total amount of incoming data during a session and the duration of that session. It helps in understanding how much bandwidth is being consumed. The value is measured in Mbit/s and lies within the range [0, IBC max]. The function $q(x) = 1 - (x / IBC_{max})$ assigns lower scores to higher bandwidth usage, indicating heavier network load.

Q: Write down FTP Operation

A:

- Client contacts FTP server at port 21
- Client obtains authorization
- Browses remote directory
- Server receives file transfer command
- Server opens TCP data connection to client
- After transfer connection closed

Q: Throughtput, goodput, badput.
A:
Throughput is the total amount of data transmitted over a network in a given time.
It includes all data, such as useful data, headers, and retransmissions.

Goodput is the amount of **useful data** received by the application per unit of time.
It does **not** include protocol overhead or retransmitted data.

Badput is the amount of data that is **not useful**, like retransmissions, errors, or dropped packets.
It reduces the overall network efficiency.

Throughput = Goodput + Badput.

Q: Factors affecting goodput.
A:

- End-to-end error rates
- Protocol functions (handshaking, windows, & acks)
- Protocol parameters (frame size, retx timers)
- pps rate of networking devices

- Lost packets at networking devices
- Workstation & server performance factors:
  - Disk-access speed
  - Disk-caching size
  - Device driver performance
- Computer bus performance (capacity/arbitration)
- Processor (CPU) performance
- Memory performance (access time for real and virtual memory)
- Operating system inefficiencies
- Application inefficiencies or bugs

- **End-to-end error rates** affect goodput because more errors mean more retransmissions, which reduce the amount of useful data received.
- **Protocol functions** like handshaking, acknowledgments, and window size also impact goodput, as they control how data is sent and confirmed.
- **Protocol parameters** such as frame size and retransmission timers can limit or improve how efficiently data is transferred.
- The **packets per second (pps) rate** of networking devices can affect goodput—devices with low pps capacity may drop or delay packets.
- **Lost packets** at routers or switches reduce goodput because lost data needs to be sent again.
- **Workstation and server performance** is also important. If disk access is slow or disk caching is small, it delays data processing.
- **Device drivers** also play a role—poorly performing drivers slow down the handling of incoming or outgoing data.
- **Computer bus performance** affects how fast data moves inside the system; slow or busy buses can create a bottleneck.
- **CPU performance** is crucial—if the processor is slow or overloaded, it can't handle data quickly.
- **Memory performance**, including access time to RAM or virtual memory, impacts how fast data can be read or written.
- **Operating system inefficiencies** (like poor scheduling or resource management) can slow down network tasks.
- Finally, **application inefficiencies or bugs** can waste resources or mishandle data, reducing the amount of useful information transferred.

Q: Easy Math Writing.
A:

Easy math writing helps make technical content clear and simple to understand.

Use the **2-3-4 rule** to keep writing clean:
Split any **sentence longer than 2 lines**, any **sentence with more than 3 verbs**, and any **paragraph with more than 4 long sentences**. This makes the text easier to read and follow.

Use **mnemonics** to remember formulas and terms. For example:

- **s** for speed
- **v** for velocity
- **t** for time

Organize your writing into **segments**.
Each segment should be **easy to read from beginning to end**, with a **clear start and finish**.

Keep segments **standalone and linear**, so the reader can understand them one at a time without needing to jump around.

This style makes math writing more accessible, especially for learning and revision.

Q: Simple Attack Analysis and Failure Probability:

A:

A Simple Attack Analysis
This analysis is based on TCP SYN flooding DoS (Denial of Service) attacks.
In this type of attack, the attacker sends **n packets** to flood the server.
The server uses a **random drop queue**, where **Q** is the **queue depth** used to manage incoming packets.

Attack Success Probability
The chance that the attack succeeds is calculated using the formula:
$\mathbf{P = 1 - (1 - 1/Q)^n}$
This shows that the more packets (n) the attacker sends, the higher the success rate.

Attack Failure Probability
This is the opposite of the success probability.
It is given by:
$\mathbf{1 - P}$

Long Questions

1. *Error Finding*:

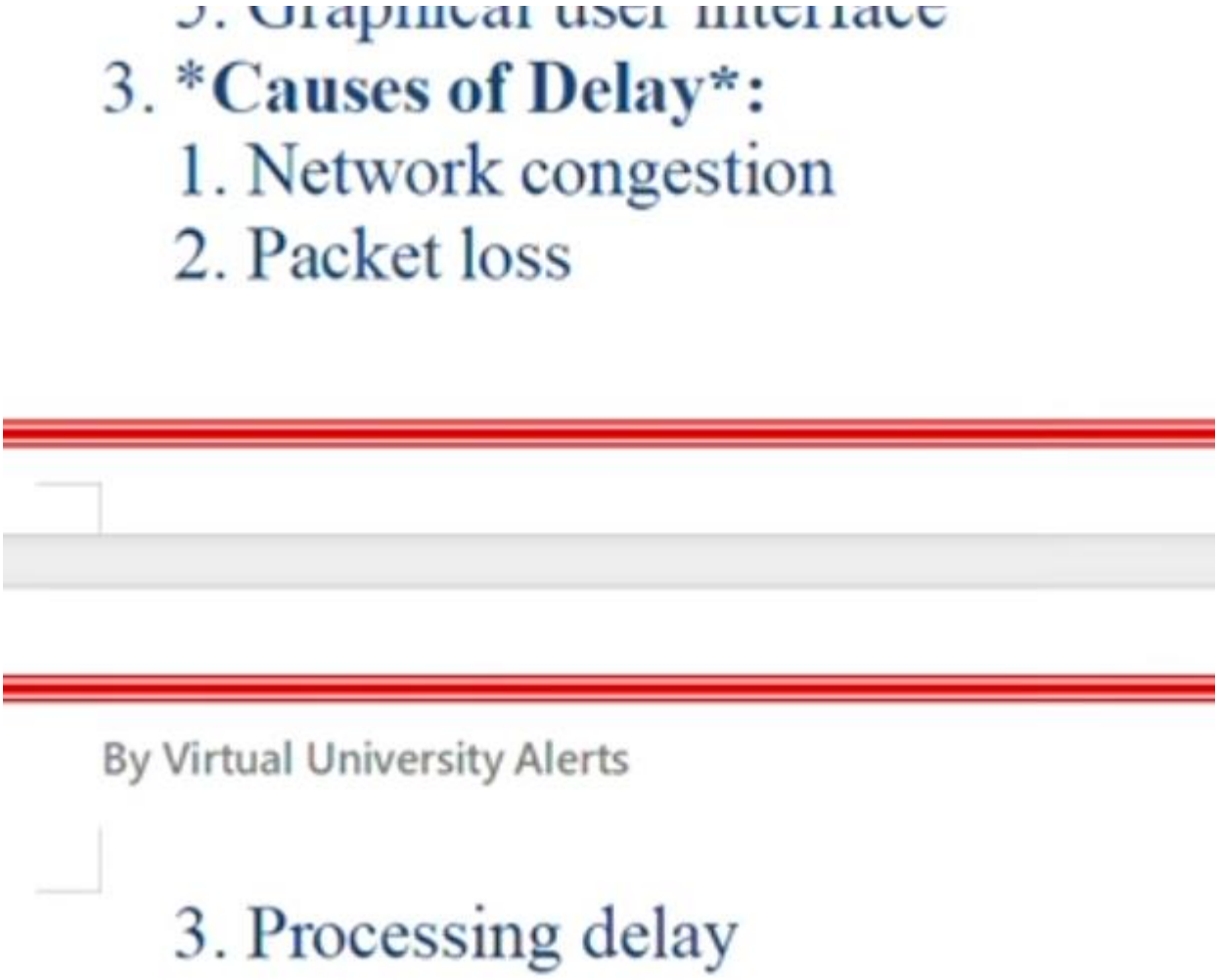
Errors can occur due to various reasons such as:

1. Incorrect assumptions
2. Inaccurate data
3. Modeling errors
4. Simulation errors
5. Human error

By Virtual University Ale

2. *OMNeT++ Features*:

1. Modular architecture
2. Support for various network protocols
3. Scalability
4. Flexibility



Q: calculate failture probability of 0.25

A:
If the success probability is **0.25**, then:
Failure Probability = 1 – 0.25 = 0.75

Q: Attack success probability example:
A:
Attack Success Probability Example

Formula:
$P = 1 – (1 – 1/Q)^n$

Given:

- Number of attack packets, **n = 5**
- Queue depth, **Q = 10**

Step-by-step Calculation:
$P = 1 – (1 – 1/10)^5$
$= 1 – (0.9)^5$
$= 1 – 0.59049$
$= \mathbf{0.40951}$

Answer:
The **attack success probability is approximately 0.41** (or 41%).

QoE—MTBF AND MTTR
Averaging out the availability

Availability as MTBF

- Mean time bw failure (MTBF) & mean time to repair (MTTR)
- Component vs service
  - Mean time bw service outage (MTBSO)
  - Mean time to recover from service outage (MTTSO)
- Typical MTTF value is once per 4000 hrs or 166.7 days
- Typical acceptable MTTR value is one hour

Availability = MTBF/(MTBF + MTTR)

4,000/4,001 = 99.98% availability

- MTBF with MTTR help to assess frequency and length of service outage
  - Mean value must be supported with variance
- The difference between MTTF and MTBF is the assumption of the former that the system shall be repaired while in the later the system is replace

Q: Simulation building process
A:
Simulation Building Process

We are simulating a **one-hop wireless communication** between two notebooks using a **WiFi Access Point (AP)**.

The main **entities** in the simulation are:

- **Wireless computers** and their **packets** (many instances)
- A single **WiFi AP**
- A **traffic generator** that creates the wireless computers and packets

The **system states** include whether the **WiFi AP is idle or busy**.

Each **wireless computer** creates several **packets**, and each packet may be **successfully delivered or fail**.

The main **events** in the simulation are:

- Creation of wireless computers
- Generation of packets
- Activity at the wireless AP

We use **event queues** to manage and schedule these events.

- Packets wait in the **output queue** of each wireless computer
- Packets also wait in the **input queue** of the WiFi AP

To make the simulation more **realistic and dynamic**, we add **randomization**.
Random values are used for:

- **Packet lengths**
- **Number of frames per wireless computer**
- **Number of wireless computers**
- **Bit Error Rate (BER)** and **Packet Error Rate (PER)**
- **Packet drop ratio** at the WiFi AP input queue

We also use **random distributions** to spread out values:

- **Uniform** or **Gaussian (Normal)** distributions can be used for packet lengths and number of frames.

This setup helps us understand how wireless networks behave under different conditions.

Five Building Steps of a Simulation

1. Identify Simulation Entities
The main entities in the simulation are wireless computers, packets (multiple instances), a WiFi Access Point (AP), and a traffic generator.
The traffic generator is responsible for creating the computers and their packets.

2. Define System States
The system states include the status of the WiFi AP (idle or busy) and the state of each packet (success or failure).
These states help track the progress of each component during the simulation.

3. Specify Events
Key events in the simulation include:

- Creation of wireless computers

- Generation of packets
- Activity at the WiFi AP (such as processing a packet)

4. Design Queues
Queues are used to schedule and manage events.
For example:

- Packets waiting in the output queue of a wireless computer
- Packets stored in the input queue of the WiFi AP before transmission

5. Add Randomization and Distributions
To make the simulation dynamic and realistic, random values are used. These include:

- Packet lengths
- Number of frames per computer
- Number of computers
- Bit Error Rate (BER), Packet Error Rate (PER), and drop ratio
  Distributions like **Uniform** or **Gaussian** are applied to model these values realistically.

Five factors of network performance:
1. Bandwidth
Bandwidth is the maximum data rate a network can handle. Higher bandwidth allows faster data transfer.

2. Latency
Latency is the time it takes for data to travel from source to destination. Lower latency means quicker response time.

3. Packet Loss
Packet loss happens when data packets are dropped due to network congestion or errors. This reduces reliability and performance.

4. Jitter
Jitter is the variation in delay between packets. High jitter affects real-time applications like video calls and online gaming.

5. Throughput
Throughput is the actual amount of useful data transmitted over time. It shows the network's efficiency in real conditions.